

Proyecto 1: Base de Datos Distribuida a Pequeña Escala

Opción B — Comercio / Inventario Multi-tienda

Henry Núñez Pérez Nicol Otárola Porras Giancarlo Vega Marín

Carnés: 2022089224, 2021117282, 2020195338

TI-4601 Bases de Datos Avanzadas

Escuela de Administración de Tecnologías de Información

Instituto Tecnológico de Costa Rica

I. REQUISITOS Y REGIONES

I-A. Dominio seleccionado

Se modela un sistema de comercio con inventario distribuido en múltiples tiendas. El sistema gestiona productos, su stock por región y los pedidos realizados en cada tienda.

I-B. Regiones simuladas

- tienda-a — Primera tienda regional.
- tienda-b — Segunda tienda regional.
- cd-central — Centro de distribución central, único nodo con permisos de escritura sobre el stock de todas las regiones.

I-C. Regla de residencia de datos

El stock de cada tienda únicamente puede ser escrito desde su propia región. La única excepción es cd-central, que puede actualizar el stock de cualquier región mediante sincronización batch documentada. Esta regla es una restricción de aplicación que se materializa en el diseño de fragmentación y en las reglas de localidad del motor.

I-D. Motor y stack

- Motor: CockroachDB (clúster de 3 nodos)
- Cliente: Python con psycopg
- Simulación de falla de sitio: `docker stop`
- Repositorio del equipo: <https://github.com/henryda2004/proyector1>

II. ENTREGABLE 1 — DISEÑO DE FRAGMENTACIÓN Y ASIGNACIÓN

II-A. Esquema lógico con PK/FK

El esquema lógico se presenta en la Tabla I y se ilustra en la Figura 1.

II-A1. producto: La clave primaria es `codProducto`, un identificador único por producto. No tiene claves foráneas ya que es la entidad raíz del esquema.

II-A2. stock: La clave primaria es compuesta por `codProducto` y `region`, ya que un producto tiene exactamente un registro de stock por región. Esto garantiza que no existan duplicados y permite la fragmentación directa por `region`.

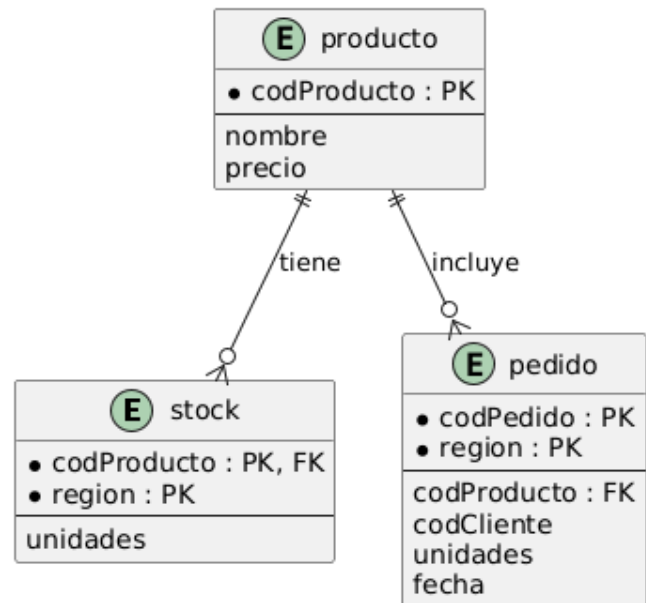


Figura 1. Diagrama lógico del esquema.

II-A3. pedido: La clave primaria es compuesta por `region` y `codPedido`. La columna `region` debe formar parte de la clave primaria porque en CockroachDB es requisito de un motor para declarar la tabla como REGIONAL BY ROW: la clave primaria determina en qué rango y en qué nodo vive físicamente cada fila [2]. Esto permite la fragmentación horizontal por región de forma primaria.

II-A4. Decisiones de simplificación: Se optó por no crear una entidad `tienda` dado que el enunciado define tres regiones fijas. El atributo `region` cumple el mismo rol con menor complejidad. El subtotal no se almacena en `pedido` ya que es un valor calculable a partir de `unidades` y `precio` en `producto`.

II-B. Fragmentación horizontal: predicados

La tabla `producto` no se fragmenta. Al ser una entidad global, un mismo producto puede existir en todas las regiones simultáneamente.

Cuadro I
ESQUEMA LÓGICO.

Relación	Atributos	Clave primaria	Claves foráneas
producto	codProducto, nombre, precio	codProducto	—
stock	codProducto, region, unidades	(codProducto, region)	codProducto → producto
pedido	codPedido, codProducto, region, codCliente, unidades, fecha	(region, codPedido)	codProducto → producto

Las tablas *stock* y *pedido* se fragmentan horizontalmente de forma primaria, ya que ambas poseen el atributo *region* propio que permite definir los predicados directamente sobre cada tabla [1].

II-B1. Fragmentos de stock:

- F_1 : Registros de stock correspondientes a la tienda A.
 $region = 'tienda-a'$
- F_2 : Registros de stock correspondientes a la tienda B.
 $region = 'tienda-b'$
- F_3 : Registros de stock del centro de distribución central.
 $region = 'cd-central'$

II-B2. Fragmentos de pedido:

- F_1 : Pedidos realizados en la tienda A.
 $region = 'tienda-a'$
- F_2 : Pedidos realizados en la tienda B.
 $region = 'tienda-b'$
- F_3 : Pedidos procesados desde el centro de distribución central.
 $region = 'cd-central'$

II-C. Tabla de verificación

Cuadro II
VERIFICACIÓN DE LA FRAGMENTACIÓN HORIZONTAL

Propiedad	¿Se cumple?	Justificación
Compleitud	Sí	Los predicados incluyen las tres regiones permitidas, donde cada registro pertenece a uno de los fragmentos definidos.
Reconstrucción	Sí	Las relaciones originales se recuperan mediante la unión de sus fragmentos, sin pérdida de registros ni atributos.
Disyunción	Sí	Los valores de <i>region</i> son mutuamente excluyentes, por lo que un registro no pertenece a dos fragmentos simultáneamente.

II-D. Réplica

Se utiliza un factor de replicación físico $RF = 3$, de manera que CockroachDB mantenga una copia de los datos en cada uno de los tres nodos. La tabla *producto* se considera global y se replica en tienda-a, tienda-b y cd-central, debido a que sus datos son consultados constantemente por las tablas *stock* y *pedido* con la finalidad de que se permita realizar las consultas y uniones con *producto* localmente, evitando accesos remotos.

Los fragmentos de *stock* y *pedido* conservan una región principal según el valor del atributo *region*, aunque CockroachDB mantiene réplicas físicas para recuperación ante

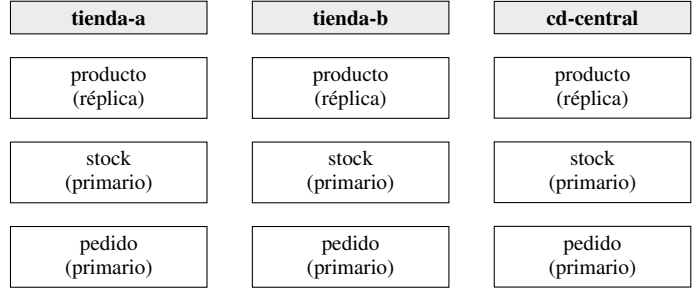


Figura 2. Diagrama de asignación de fragmentos a nodos.

Cuadro III
ASIGNACIÓN DE FRAGMENTOS A NODOS.

Fragmento	Nodo	Tipo
producto	tienda-a	réplica
producto	tienda-b	réplica
producto	cd-central	réplica
stock (tienda-a)	tienda-a	primario
stock (tienda-b)	tienda-b	primario
stock (cd-central)	cd-central	primario
pedido (tienda-a)	tienda-a	primario
pedido (tienda-b)	tienda-b	primario
pedido (cd-central)	cd-central	primario

fallas. El factor $RF = 3$ permite que el sistema continúe funcionando ante la caída de un nodo, por ende, el costo de esta decisión es un mayor consumo de almacenamiento y comunicación durante las escrituras, pero se obtiene mayor disponibilidad y tolerancia a fallos.

II-E. Descripción de la asignación

Cada fragmento de *stock* y *pedido* se asigna al nodo de la región que lo originó, garantizando que las operaciones de lectura y escritura locales no requieran comunicación entre nodos. La tabla *producto* se replica en los tres nodos por ser de consulta global. La asignación completa se detalla en la Tabla III.

II-E1. Sincronización batch de cd-central: El nodo *cd-central* es el único autorizado a escribir sobre fragmentos de *stock* que pertenecen a otras regiones. Esta escritura ocurre de forma periódica y programada (por ejemplo, una vez al día), no en tiempo real, y su propósito es reabastecer el inventario reportado por cada tienda desde el centro de distribución. Al tratarse de una escritura que cruza la frontera de región, esta operación se clasifica como una escritura que

Cuadro IV
NODOS DEL CLÚSTER Y SUS REGIONES.

Contenedor	Región	Rol
crdb-1	tienda-a	Tienda regional
crdb-2	tienda-b	Tienda regional
crdb-3	cd-central	Centro de distribución

cruza región en las mediciones del Entregable 3. La restricción de que solo `cd-central` puede realizar esta escritura es una regla de aplicación: CockroachDB no impide técnicamente que cualquier nodo escriba sobre cualquier fragmento, por lo que el cumplimiento de esta regla depende de que el cliente (la aplicación) respete esta política.

III. ENTREGABLE 2: IMPLEMENTACIÓN

III-A. Entorno

El sistema se implementó sobre el clúster del laboratorio 1: tres nodos de CockroachDB v24.3.0 levantados con Docker Compose, cada uno con una localidad de región distinta. Los nombres de las regiones se ajustaron al dominio del proyecto, como se muestra en la Tabla IV.

III-B. Cómo reproducir

Desde la raíz del repositorio, en una máquina con Docker y Docker Compose, se ejecuta:

```
./scripts/setup.sh
```

El script realiza de forma automática los siguientes pasos:

1. Elimina contenedores y volúmenes previos para partir de un estado limpio.
2. Levanta los tres nodos de CockroachDB.
3. Espera a que el clúster esté disponible.
4. Configura la base `ti4601` como multi-región (`sql/regions.sql`).
5. Crea las tablas con su localidad (`sql/schema.sql`).
6. Carga los datos de prueba (`sql/seed.sql`): 10 productos, 30 filas de stock y 300 pedidos.
7. Verifica la carga y genera la evidencia en la carpeta `evidence/`.

Las instrucciones completas se encuentran en el `README.md` del repositorio.

III-C. Localidad de las tablas

La salida de `SHOW CREATE TABLE` (Figura 3) confirma que el diseño del Entregable 1 quedó implementado en el motor: `producto` utiliza `LOCALITY GLOBAL`, mientras que `stock` y `pedido` utilizan `LOCALITY REGIONAL BY ROW AS region`. En ambas tablas regionales la columna `region` forma parte de la clave primaria: (`region`, `codproducto`) en `stock` y (`region`, `codpedido`) en `pedido`.

```
producto:
  PRIMARY KEY (codproducto ASC)
  LOCALITY GLOBAL

stock:
  PRIMARY KEY (region ASC, codproducto ASC)
  LOCALITY REGIONAL BY ROW AS region

pedido:
  PRIMARY KEY (region ASC, codpedido ASC)
  LOCALITY REGIONAL BY ROW AS region
```

Figura 3. Extracto de `SHOW CREATE TABLE` (`evidence/show_create.txt`).

Cuadro V
EXTRACTO DE `SHOW RANGES`.

Tabla	Rangos	Réplicas	Votantes
producto	70	{1,2,3}	{1,2,3}
stock	71 a 75	{1,2,3}	{1,2,3}
pedido	75 a 79	{1,2,3}	{1,2,3}

region	count
cd-central	100
tienda-a	100
tienda-b	100

Figura 4. Pedidos por región (`evidence/verification.txt`).

III-D. Distribución de rangos

La salida de `SHOW RANGES` (Tabla V) muestra cómo el motor dividió cada tabla. La tabla `pedido` quedó en cinco rangos: tres contienen los datos, uno por cada región, y dos son rangos límite sin datos. Todos los rangos de `producto`, `stock` y `pedido` tienen tres réplicas votantes {1, 2, 3}, una en cada región, lo que coincide con el factor de replicación $RF = 3$ descrito en el Entregable 1.

III-E. Verificación de la carga

La consulta de verificación (Figura 4) confirma que los 300 pedidos quedaron distribuidos de forma uniforme: 100 en `tienda-a`, 100 en `tienda-b` y 100 en `cd-central`. Esto demuestra que cada fila fue asignada a su fragmento según el valor de `region`.

IV. ENTREGABLE 3: MEDICIONES

IV-A. Metodología

Se considera una operación *local* cuando el cliente se conecta al nodo de `tienda-a` (R1) y accede a filas cuya región hogar es `tienda-a`; se considera que *cruza región* cuando, desde ese mismo nodo, accede a filas cuya región hogar es `tienda-b` (R2).

Se midieron cuatro operaciones sobre la tabla `stock` con el script `scripts/measure_e3.py`:

- Lectura local: `SELECT` de una fila con `region = 'tienda-a'`.
- Lectura remota: `SELECT` de una fila con `region = 'tienda-b'`.

Cuadro VI
LATENCIAS MEDIDAS (MS).

Operación	Desde	p50	p99	n
Lectura local	R1	2.40	2.80	30
Lectura remota	R1 → R2	2.43	3.09	30
Escritura local	R1	26.62	99.35	30
Escritura cruza región	R1 → R2	28.86	63.73	30

- Escritura local: UPDATE de una fila con `region = 'tienda-a'`.
- Escritura que cruza región: UPDATE de una fila con `region = 'tienda-b'`.

Condiciones de la medición:

- Los tres nodos corren en la misma máquina (CPU: AMD Ryzen 5 8645HS, RAM: 16), con los mismos límites de recursos para todos.
- Se usó el perfil completo del laboratorio 1 (`cache` y `max-sql-memory` de 256 MiB por nodo).
- Se descartaron 10 corridas de calentamiento por caso para eliminar el efecto de arranque en frío.
- Se registraron 30 corridas válidas por caso.
- Se reportan la mediana (p50) y el percentil 99 (p99), calculados por interpolación lineal.
- No se utilizó latencia inyectada entre nodos.

La escritura que cruza región corresponde, en el dominio, a la sincronización batch de `cd-central`, la única operación autorizada a escribir stock de otra región. Se mide desde `tienda-a` hacia `tienda-b` para mantener el formato R1 hacia R2 solicitado en el enunciado.

IV-B. Resultados

Los resultados se presentan en la Tabla VI y se encuentran en `evidence/e3_metrics.csv`.

IV-C. Interpretación

Las lecturas local y remota son prácticamente iguales (p50 de 2.40 ms y 2.43 ms). Esto es esperable: las regiones son lógicas y los tres nodos se comunican a través de la red interna de Docker en la misma máquina, por lo que no existe la distancia física que tendría un despliegue real entre regiones. Aun así, el p99 de la lectura remota (3.09 ms) es mayor que el de la local (2.80 ms), ya que la lectura remota debe ir al leaseholder ubicado en otro nodo, lo que agrega un salto de red adicional.

Las escrituras son aproximadamente diez veces más lentas que las lecturas. La razón es que toda escritura, incluso la local, debe confirmarse mediante consenso Raft: el leaseholder necesita que una mayoría de réplicas (2 de 3) acepte el cambio antes de responder al cliente, lo que implica comunicación entre nodos en ambos casos [3].

El p99 de la escritura local (99.35 ms) resultó mayor que el de la escritura que cruza región (63.73 ms). Con 30 corridas, el p99 queda determinado prácticamente por el valor más alto de la muestra, por lo que es muy sensible a picos aislados de la máquina anfitriona y no debe interpretarse como que escribir

Cuadro VII
BITÁCORA DE LA PRUEBA DE FALLA.

Hora (UTC)	Evento
16:47:40.094	Primera escritura confirmada (estado sano)
16:47:48.099	Última escritura confirmada antes de la falla
16:47:48	Se detiene <code>crdb-2</code>
16:47:56.353	Intento fallido: <i>statement timeout</i>
16:48:00.384	Primera escritura confirmada tras la falla
16:48:40.322	Última escritura confirmada de la prueba

localmente sea más lento. Las medianas, que son más estables, sí muestran el orden esperado (26.62 ms frente a 28.86 ms).

Para observar el costo real de cruzar regiones sería necesario inyectar latencia controlada entre los nodos, lo cual queda fuera del alcance de esta medición.

V. ENTREGABLE 4: FALLA DE SITIO

V-A. Preparación

Los rangos de las tablas `REGIONAL BY ROW` no son adecuados para esta prueba, ya que en un clúster con un solo nodo por región pueden quedar subreplicados, como advierte la guía del laboratorio 1 [4]. Por esto se creó una tabla de control independiente, `e4_probe`, en la base `ti4601_e4` (`sql/e4_probe.sql`), configurada con tres réplicas votantes (`num_replicas = 3`, `num_voters = 3`), una en cada nodo. La tabla contiene una única fila con un contador `version` que aumenta en uno con cada escritura confirmada.

Antes de la prueba se verificó que el rango de la tabla tuviera las réplicas `{1, 2, 3}` y se movió su lease al nodo de `tienda-b`, que es el nodo que se detiene. Así se evalúa el peor caso: la caída del nodo que coordina las escrituras.

V-B. Protocolo

La prueba se ejecutó con una sonda de escrituras (`scripts/e4_probe.py`), basada en la sonda del laboratorio 1. La sonda realiza una escritura cada 0.5 segundos durante 60 segundos y registra la hora, el resultado y la latencia de cada intento en `evidence/e4-chaos.csv`. El protocolo fue:

1. Estado sano: la sonda confirma escrituras con normalidad.
2. Se detiene el contenedor `crdb-2` con `docker compose stop`.
3. Se registra el instante de la detención y se cronometra hasta la primera escritura confirmada.
4. Se verifica que la última escritura confirmada siga almacenada.
5. Se vuelve a levantar el nodo.

V-C. Bitácora

La Tabla VII resume los eventos principales, tomados de `evidence/e4_stop.txt`, `evidence/e4_probe.txt` y `evidence/e4-rpo.txt`. Las horas están en UTC.

En total la sonda realizó 29 intentos: 17 escrituras confirmadas antes de la falla, 1 intento fallido y 11 escrituras confirmadas después de la falla.

V-D. RTO

El RTO observado se calcula como el tiempo entre la detención del nodo y la primera escritura confirmada después de ella:

$$RTO = 12,0 \text{ s}$$

Este tiempo incluye la detección de la caída del nodo, la elección de un nuevo líder entre los dos nodos restantes y los reintentos del cliente. El sistema pudo seguir confirmando escrituras porque los dos nodos que quedaron activos forman una mayoría (2 de 3) del grupo Raft [3].

Se observa además que, después de la falla, cada escritura tardó cerca de 4 segundos. Este aumento se atribuye al cliente: la sonda abre una conexión nueva en cada intento con la lista de hosts `crdb-1`, `crdb-2`, `crdb-3`, y cada intento de conexión pierde tiempo con el nodo detenido. No afecta el RTO, que se mide hasta la primera escritura confirmada.

V-E. RPO

Para el RPO es necesario diferenciar un intento fallido de un commit perdido. Un intento fallido es una escritura para la cual el cliente nunca recibió confirmación; no constituye pérdida de datos, ya que al cliente no se le garantizó nada. Un commit perdido es una escritura que el cliente recibió como confirmada y que después no se encuentra en la base.

En la prueba hubo 1 intento fallido y 0 commits perdidos. La verificación posterior (`evidence/e4-rpo.txt`) muestra la fila con `version = 223` y `updated_at = 16:48:40.312`, que coincide con la última escritura confirmada por la sonda (16:48:40.322). Por lo tanto, ninguna escritura confirmada se perdió:

$$RPO = 0$$

Esto es consecuencia del consenso Raft: una escritura solo se confirma cuando la mayoría de las réplicas la guardó, de modo que la caída de un solo nodo no puede eliminar datos confirmados.

VI. ENTREGABLE 5: ¿HACÍA FALTA DISTRIBUIR?

VI-A. Alternativa evaluada

La alternativa a la base de datos distribuida es un despliegue con un nodo primario, que recibe todas las escrituras, y una réplica de lectura que copia los datos del primario y puede atender consultas. A continuación se comparan ambas arquitecturas en latencia, disponibilidad, complejidad operativa y costo en la misma máquina, utilizando los resultados obtenidos en los entregables anteriores.

VI-B. Latencia

Las lecturas del clúster distribuido tuvieron una mediana de 2.40 ms en el caso local y de 2.43 ms en el remoto (Tabla VI). La lectura local es representativa de lo que obtendría la alternativa, ya que en ambos casos un único nodo resuelve la consulta. Por lo tanto, en este entorno la distribución no aporta ventaja de latencia en lectura.

En escritura la comparación es desfavorable para el clúster distribuido. Cada escritura requiere la confirmación de una mayoría de réplicas mediante Raft, lo que explica las medianas de 26.62 ms y 28.86 ms. Un primario con réplica asíncrona confirma la escritura sin esperar a la réplica, por lo que se espera una latencia de escritura menor. Esta latencia no se midió en el proyecto, pero la diferencia se deriva del mecanismo: el primario no necesita comunicarse con otro nodo para confirmar.

La ventaja de latencia de la distribución aparecería en un despliegue real, con tiendas en ubicaciones geográficas distantes. Con un único primario, cada tienda pagaría en cada escritura la latencia de red hasta la ubicación del primario; con `REGIONAL BY ROW`, cada tienda tiene el leaseholder de sus filas en su propia región. Como en este proyecto las tres regiones son lógicas y se ejecutan en la misma máquina, esa ventaja no se pudo observar.

VI-C. Disponibilidad

La ventaja clara del clúster distribuido es la disponibilidad. En la prueba de falla del Entregable 4 se detuvo el nodo que coordinaba las escrituras y el sistema volvió a confirmar escrituras en 12.0 s, sin intervención manual. De 29 intentos, hubo 1 intento fallido y ningún commit perdido, por lo que el RPO fue cero.

En la alternativa, la caída del primario detiene todas las escrituras hasta que la réplica sea promovida a primario, operación que normalmente requiere intervención manual o herramientas adicionales. Además, si la replicación es asíncrona, las escrituras confirmadas por el primario que todavía no llegaron a la réplica se pierden, por lo que el RPO puede ser mayor que cero. Si la replicación es síncrona se evita esa pérdida, pero las escrituras se bloquean cuando la réplica deja de responder.

VI-D. Complejidad operativa

El clúster distribuido resultó considerablemente más complejo de operar. Fue necesario configurar la base como multi-región, declarar la localidad de cada tabla, incluir la región en la clave primaria de las tablas regionales y crear una tabla de control independiente para la prueba de falla, ya que con un solo nodo por región los rangos `REGIONAL BY ROW` pueden quedar subreplicados. Interpretar los resultados también exigió comprender el funcionamiento de los leaseholders y del consenso Raft. La alternativa se configura con una instancia y una réplica, sin decisiones de localidad.

VI-E. Costo en la misma máquina

Ambas arquitecturas se ejecutan sobre el mismo hardware. El clúster distribuido utiliza tres contenedores con 256 MiB de caché y 256 MiB de memoria SQL cada uno, frente a dos procesos en la alternativa. Como los tres nodos comparten la misma CPU y memoria, la distribución no agrega capacidad de cómputo: reparte el mismo recurso entre más procesos y añade el tráfico de consenso entre ellos.

Cuadro VIII
COMPARACIÓN DE ARQUITECTURAS.

Eje	Clúster distribuido	Primario + réplica
Lectura	2.40 ms local, 2.43 ms remota	Similar a la lectura local
Escritura	26.62 ms, requiere mayoría Raft	Menor, sin espera de consenso
Falla de nodo	Recuperación automática en 12.0 s, RPO = 0	Promoción manual, RPO mayor que cero si es asíncrona
Complejidad	Alta	Baja
Costo	3 nodos en la misma máquina	2 procesos en la misma máquina

VI-F. Residencia de datos

La residencia no justifica por sí sola la distribución. La regla del enunciado restringe desde qué región se escribe el stock, y esa restricción se cumple en la aplicación, lo cual también sería posible con un único primario. Además, REGIONAL BY ROW ubica el leaseholder de cada fila en su región hogar, pero las réplicas físicas se encuentran en los tres nodos (Tabla V), de modo que los datos de una tienda no permanecen únicamente en su región.

VI-G. Conclusión

La comparación se resume en la Tabla VIII. La distribución está **justificada**, pero por disponibilidad y no por latencia. En este entorno, con las tres regiones en una sola máquina, el clúster distribuido no mejora la latencia de lectura, empeora la de escritura y es más complejo de operar. Su ventaja es que tolera la caída de un nodo de forma automática y sin perder escrituras confirmadas, algo que la alternativa de primario y réplica no garantiza. En una cadena de tiendas, donde detener las ventas por la caída de un servidor tiene un costo directo, esa garantía justifica la complejidad adicional.

APÉNDICE

Scripts y evidencia

Todo el código y la evidencia se encuentran en el repositorio del equipo:

- `scripts/setup.sh`: levanta el clúster, crea el esquema, carga los datos y genera la evidencia del Entregable 2.
- `scripts/measure_e3.sh` y `scripts/measure_e3.py`: ejecutan las mediciones del Entregable 3.
- `scripts/e4_probe.py` y `sql/e4_probe.sql`: sonda y tabla de control de la prueba de falla del Entregable 4.
- `sql/regions.sql`, `sql/schema.sql` y `sql/seed.sql`: configuración multi-región, esquema y datos de prueba.
- `evidence/`: salidas de todas las ejecuciones citadas en este informe.

Roles del equipo

- **Henry Núñez Pérez**: diseño de fragmentación y asignación (Entregable 1), creación del repositorio, del esquema, los datos de prueba y la configuración multi-región, análisis comparativo de la arquitectura (Entregable 5) e integración del informe.
- **Giancarlo Vega Marín**: implementación y automatización del despliegue del clúster y generación de su evidencia (Entregable 2), y desarrollo conjunto de las mediciones de latencia (Entregable 3).
- **Nicol Otárola Porras**: desarrollo conjunto de las mediciones de latencia (Entregable 3), y diseño, ejecución y análisis de la prueba de falla de sitio (Entregable 4).

REFERENCIAS

- [1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham: Springer, 2020.
- [2] Cockroach Labs, "Multi-Region Capabilities Overview," CockroachDB Docs, v24.3.
- [3] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proc. USENIX Annual Technical Conference (ATC)*, 2014, pp. 305–319.
- [4] M. Zumbado, "Laboratorio 1: Clúster CockroachDB, localidad y quórum," TI-4601 Bases de Datos Avanzadas, Instituto Tecnológico de Costa Rica, 2026.